Interconnect Modeling for Performance Analysis of HW/SW Coherence Protocols

Bolaños Lopez Randall Bryan
email: yitanr@estudiantec.cr
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica

Molina Redondo Andres email: andresmolina@estudiantec.cr
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica

Rodríguez Cordero Christopher David
email: chrodriguez@estudiantec.cr
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica

***Abstract*—This paper presents the design and implementation of a SystemC-based simulation model of a cache-coherent interconnect for a shared-memory multiprocessor system composed of four processing elements (PEs), each with a private L1 cache. The simulator integrates a software layer—parallel workload memory traces—with a hardware layer comprising a formal bus transaction model, parametric L1 cache logic, and two cache coherence protocols: MSI (Write-Invalidate family) and Firefly (Write-Update family). A formal set of five bus transactions is defined: `BusRead`, `BusReadX`, `BusUpgrade`, `BusUpdate`, and `BusInvalidate`. Two parallel workloads are evaluated—Blocked Matrix Multiplication under high contention and a Cryptography/Network Pipeline under a producer-consumer pattern—plus an exploratory migratory access scenario. Results show that under high contention MSI generates 39 invalidations with zero updates, while Firefly replaces them with 114 broadcast updates at equal total transaction count. In the pipeline scenario, Firefly reduces total bus transactions by 22% (18 to 14) by proactively updating remote caches and avoiding re-fetches. The migratory pattern shows a 41% transaction reduction under Firefly. The simulation exports VCD waveforms, per-cache statistics, and CSV data for reproducible analysis.**

***Index Terms*—cache coherence, MSI protocol, Firefly protocol, SystemC, interconnect modeling, shared memory, multiprocessor, bus transactions, Write-Invalidate, Write-Update**

I. Introducción

Modern multiprocessor systems rely on shared-memory architectures in which multiple processing cores communicate through a common interconnect. Ensuring that all processors observe a consistent view of shared data is the fundamental challenge addressed by cache coherence protocols [**?**]. The choice of protocol has a direct impact on bus traffic, memory bandwidth consumption, and overall system performance.

Two dominant protocol families exist. Write-Invalidate protocols, such as MSI [**?**], respond to a write by invalidating all remote copies of the modified block, forcing subsequent readers to re-fetch it. Write-Update protocols, such as Firefly [**?**], instead broadcast the new value to all sharers, keeping their copies valid at the cost of higher per-write bus traffic.

This project implements a cycle-accurate SystemC [**?**] simulation of a four-PE shared-memory system with private L1 caches and a cache-coherent bus. The simulator integrates memory traces generated by parallel workloads with the hardware model, enabling a direct comparison of MSI and Firefly under realistic access patterns. A key contribution of this work is the formalization of a typed bus transaction set that makes the distinction between `BusReadX` (write miss from I) and `BusUpgrade` (write hit from S) explicit, improving both correctness and observability of the simulation.

The remainder of this paper is organized as follows. Section **??** provides the theoretical background. Section **??** describes the system architecture and design decisions. Section **??** presents and analyzes the experimental results. Section **??** concludes.

II. Theoretical Background

A. Cache Coherence

In a shared-memory multiprocessor, each processor has one or more levels of private cache. When two processors hold copies of the same memory block and one modifies it, the other's copy becomes stale. Cache coherence protocols enforce the invariant that all processors observe a consistent value for every memory location [**?**], [**?**]. A coherence protocol defines stable states for each cache block and transitions triggered by local processor requests or bus snooping events [**?**].

B. MSI Protocol

MSI is the baseline Write-Invalidate protocol [**?**]. Each cache block resides in one of three stable states:

- **Modified (M):** Present only in this cache; written locally. The cache holds exclusive ownership.
- **Shared (S):** Clean copy; may be present in multiple caches.
- **Invalid (I):** Not usable; must be fetched before use.

A write from state I issues `BusReadX`, fetching the block and invalidating all remote copies. A write from state S issues `BusUpgrade`, which invalidates remote copies without re-transferring data—an optimization that reduces unnecessary bus traffic [**?**].

C. Firefly Protocol

Firefly [**?**] is a Write-Update protocol. On a write, the interconnect broadcasts the new value to all caches that hold the block via a `BusUpdate` transaction. All sharers remain

in state S. If no sharer exists, the writing cache gains exclusive ownership (state M) without broadcasting. This approach eliminates future read misses for existing sharers at the cost of additional traffic per write [**?**].

D. Bus Transaction Formalization

A bus transaction is a typed message emitted by a cache to the interconnect, specifying both the requested operation and the required coherence action. Table **??** defines the five transactions implemented in this work.

TABLE I
FORMAL BUS TRANSACTION SET

Transaction	Description and State Transition
`BusRead`	Read miss (I→S). Fetches block from memory or a downgraded M-owner.
`BusReadX`	Write miss from I (I→M). Fetches block; invalidates all remote copies.
`BusUpgrade`	Write hit from S (S→M). Invalidates remote copies; no data transfer.
`BusUpdate`	Firefly write. Broadcasts new value to all sharers; they remain in S.
`BusInvalidate`	Internal signal emitted by the interconnect to force I on remote caches.

E. False Sharing

False sharing occurs when processors access distinct data items that share a cache block [**?**]. Under Write-Invalidate, a write to any byte invalidates the entire block in all other caches, even if other processors accessed different bytes. This is the primary source of coherence overhead in the matrix multiplication workload.

III. SYSTEM DESIGN

A. Overall Architecture

The simulated system consists of four PEs connected to a shared cache-coherent bus, each with a private L1 cache, backed by shared main memory [**?**]. The entire hardware model is implemented in SystemC/C++17 [**?**]. Python is used only for post-processing and plot generation.

B. Processing Elements

Each PE is an `SC_THREAD` that reads a memory trace file and issues `READ` or `WRITE` requests to its local L1 cache. Trace lines follow the format `<pe_id> <READ|WRITE> <hex_address>`. Each PE filters only lines assigned to its identifier, enabling all four PEs to execute in parallel from the same file.

C. L1 Cache Design

The L1 cache is a parametric set-associative module supporting LRU and FIFO replacement. Table **??** shows the default configuration used for all experiments. This configuration was chosen to ensure sufficient evictions and conflict misses to stress the coherence protocol [**?**], [**?**].

TABLE II
L1 CACHE DEFAULT CONFIGURATION

Parameter	Value
Total size	1 024 bytes
Block size	64 bytes
Associativity	2-way
Replacement policy	LRU
L1 hit latency	1 ns
L1 miss detect latency	1 ns

The `access(op, addr)` method determines whether the block is present and valid, tracks hit/miss/eviction/conflict-miss counters, and delegates to the interconnect when a bus transaction is required.

D. Interconnect and Transaction Model

The interconnect exposes `issue_transaction(tx, id, addr)` as its primary interface. The cache selects the transaction type based on the current block state and the active protocol:

- Read miss ⇒ `BusRead`
- Write from I, MSI ⇒ `BusReadX`
- Write from S, MSI ⇒ `BusUpgrade`
- Any write, Firefly ⇒ `BusUpdate`

Internally, each transaction type is handled by a dedicated method (`do_bus_read`, `do_bus_read_x`, etc.), separating concerns and making the protocol logic auditable. The interconnect timing model uses four configurable parameters: bus arbitration (1 ns), memory latency (5 ns), shared cache latency (2 ns), and coherence overhead (2 ns).

a) Arbitraje del Bus (FCFS): El interconect incluye un arbitro interno que implementa una política first-come, first-served (FCFS). Las caches encolan sus solicitudes en una estructura de peticiones dentro de `Interconnect` y un `SC_THREAD` de concesión procesa la cola, otorgando el bus a una única petición por vez. Esta implementación proporciona un ordenamiento estricto FCFS y facilita el análisis de fairness y latencias de espera.

E. Workloads

Workload 1 – Blocked Matrix Multiplication: Four PEs compute sub-blocks in parallel and write to a shared accumulator address. All PEs write to the same cache block, maximizing false sharing and coherence contention.

Workload 2 – Cryptography/Network Pipeline: A four-stage pipeline where PE0 writes a packet, PE1 reads and rewrites, PE2 reads, and the pattern repeats for multiple packets. This models a unidirectional producer-consumer migration.

Exploratory – Migratory Pattern: A single shared token address is read and written by each PE in round-robin, representing the worst case for Write-Invalidate protocols.

IV. Results and Analysis

A. Workload 1: Blocked Matrix Multiplication

Table **??** summarizes bus statistics for this high-contention workload.

TABLE III
Bus Statistics – Blocked Matrix Multiplication

Metric	MSI	Firefly
Total transactions	44	44
BusRead	4	4
BusReadX	40	0
BusUpgrade	0	0
BusUpdate	0	40
Invalidations	39	0
Broadcast updates	0	114
Downgrades M→S	0	0
Simulation time (ns)	91	91

Under MSI, every write to the shared address issues `BusReadX`, producing 39 invalidations. Each PE must then re-fetch the block before its next write, creating a ping-pong ownership transfer between all four caches—a classic invalidation storm [**?**]. Under Firefly, all 40 writes use `BusUpdate`, producing 114 individual remote cache updates (up to 3 per write). No invalidations occur and no re-fetches are needed. The total transaction count is equal (44), but the coherence mechanism is entirely different. Fig. **??** and Fig. **??** illustrate this complementary behavior.

B. Workload 2: Cryptography/Network Pipeline

Table **??** shows results for the producer-consumer workload.

TABLE IV
Bus Statistics – Cryptography/Network Pipeline

Metric	MSI	Firefly
Total transactions	18	14
BusRead	8	4
BusReadX	6	0
BusUpgrade	4	0
BusUpdate	0	10
Invalidations	12	0
Broadcast updates	0	17
Downgrades M→S	6	1
Simulation time (ns)	46	46

A notable result is the appearance of 4 `BusUpgrade` transactions in MSI. PE1 reads a block (entering state S) before writing it in the next stage. Since it already holds valid data, only an upgrade is issued—no data re-transfer occurs. This confirms the importance of distinguishing S→M upgrades from I→M read-exclusive requests in the transaction model.

Firefly reduces total transactions from 18 to 14 (22%). Proactive updates allow downstream consumers to find data already valid in their caches, eliminating four `BusRead` transactions that MSI requires after invalidation. This is consistent with the theoretical advantage of Write-Update in producer-consumer patterns [**?**], [**?**].

C. Exploratory: Migratory Access Pattern

Table **??** shows the migratory scenario results.

TABLE V
Bus Statistics – Migratory Access Pattern

Metric	MSI	Firefly
Total transactions	61	36
Invalidations	55	0
Broadcast updates	0	96
Downgrades M→S	16	0
Simulation time (ns)	91	73

This pattern is the worst case for MSI: every write invalidates all others, forcing re-fetches before the next read. MSI generates 61 transactions vs. 36 for Firefly—a 41% reduction—and finishes 18 ns sooner, consistent with classical analysis of migratory patterns [**?**], [**?**]. Fig. **??** and Fig. **??** visualize these differences across all three workloads.

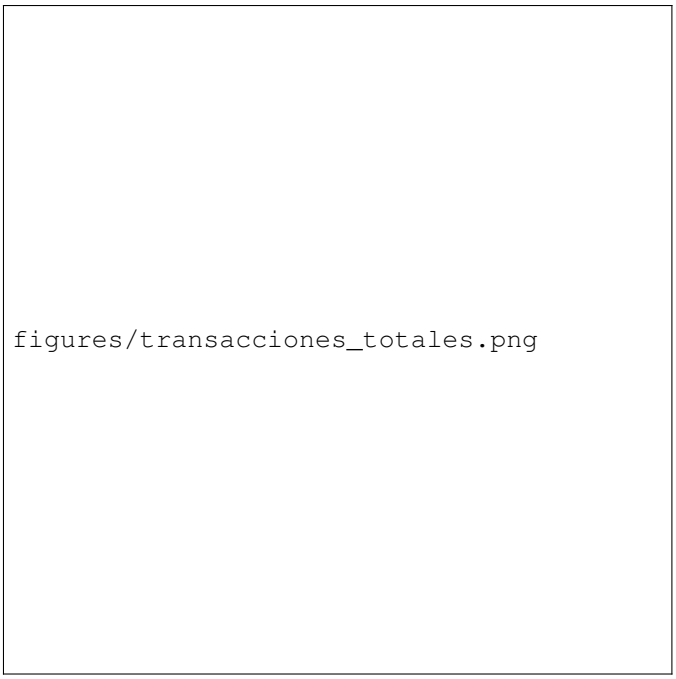


Fig. 1. Total bus transactions per workload and protocol.

V. Conclusions

This work demonstrates that cache coherence protocol selection has a significant, workload-dependent impact on bus traffic and execution time. Key findings are:

1) **MSI generates invalidation storms under high contention.** In the matrix workload, 89% of bus transactions (39/44) are invalidations that force re-fetches, confirming the classical weakness of Write-Invalidate under false sharing [**?**].

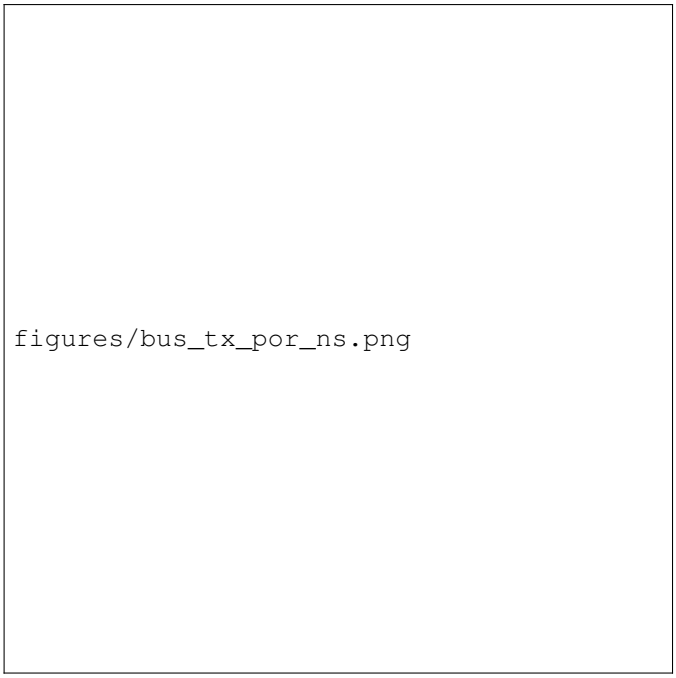


Fig. 2. Approximate bus bandwidth (tx/ns) per workload and protocol.

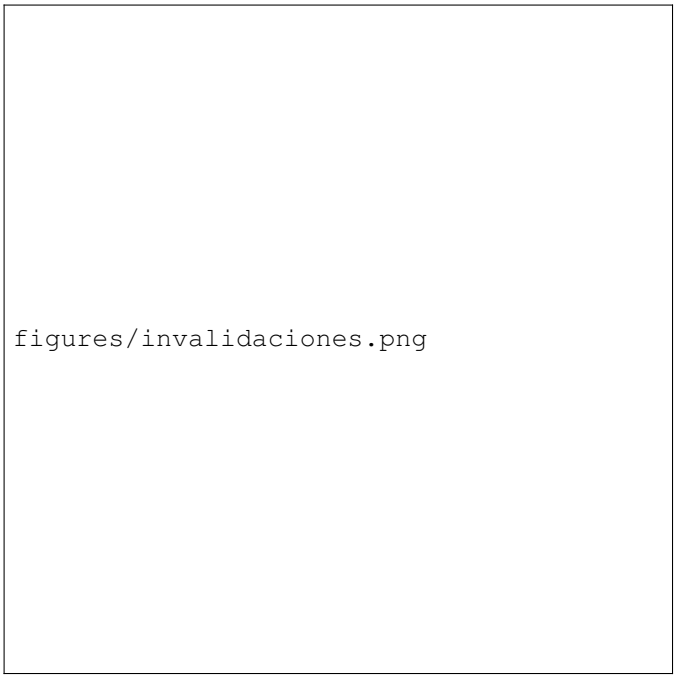


Fig. 3. Invalidations: MSI vs. Firefly (Firefly always zero).



Fig. 4. Broadcast updates: MSI vs. Firefly (MSI always zero).

2) **Firefly trades invalidations for broadcast updates.** Under high contention, Firefly eliminates all invalidations and replaces them with 114 updates. Whether this is beneficial depends on the subsequent read rate from the updated caches.
3) **Formalizing BusUpgrade vs. BusReadX improves correctness.** The pipeline workload shows 4 `BusUpgrade` transactions that would have been incorrectly counted as `BusReadX` without explicit state-based transaction selection, avoiding spurious data transfers.
4) **Firefly outperforms MSI in producer-consumer and migratory patterns.** Transaction reductions of 22% and 41% respectively confirm the theoretical predictions of [**?**], [**?**].
5) **No protocol dominates all workloads.** Protocol selection must be informed by the expected sharing pattern: Write-Update favors producer-consumer and migratory access; Write-Invalidate may be preferable for predominantly private or write-once workloads.

REFERENCES

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Elsevier, 2017.
[2] V. Nagarajan, D. J. Sorin, M. D. Hill, and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed. Morgan & Claypool, 2020.
[3] J. Archibald and J.-L. Baer, "Cache coherence protocols: Evaluation using a multiprocessor simulation model," *ACM Trans. Comput. Syst.*, vol. 4, no. 4, pp. 273–298, Nov. 1986.
[4] C. P. Thacker, L. C. Stewart, and E. H. Satterthwaite Jr., "Firefly: A multiprocessor workstation," *IEEE Trans. Comput.*, vol. 37, no. 8, pp. 909–920, Aug. 1988.
[5] IEEE, *IEEE Standard for Standard SystemC Language Reference Manual*, IEEE Std 1666-2023, 2024.
[6] IEEE, *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[7] R. García Fernández and L. Chavarría Zamora, "CE 4302 Proyecto Grupal 01: Modelado de Interconnect para el análisis de desempeño de protocolos de coherencia HW/SW," Instituto Tecnológico de Costa Rica, Cartago, Costa Rica, Mar. 2026.
[8] Y. Solihin, *Fundamentals of Parallel Multicore Architecture*. CRC Press, 2016.
[9] D. Culler, J. P. Singh, and A. Gupta, *Parallel Computer Architecture: A Hardware/Software Approach*. Morgan Kaufmann, 1999.
[10] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Elsevier, 2013.